ReviewLens

Aspect-Level Sentiment Analysis for Turkish E-Commerce Reviews

Course: Natural Language Processing
Track: Track 1 — Natural Language Processing (NLP)
Student: Kerem Özcan
Date: April 2026

Code Repository: https://github.com/KeremOzcn/reviewlens
Video Presentation: https://youtu.be/XgDiJUnKe2Y

---

1. Problem Statement

Online shopping platforms accumulate thousands of product reviews daily. Reading through them manually is impractical for both buyers making purchase decisions and sellers trying to improve their products. A simple star rating does not capture *why* customers are satisfied or dissatisfied — is it the product quality, shipping speed, price fairness, or customer service?

This project builds **ReviewLens**, an intelligent NLP application that:

- Reads a list of Turkish product reviews
- Understands the sentiment of each review at a sentence level using a fine-tuned Transformer model
- Assigns separate sentiment scores to 6 product aspects (quality, shipping, price, durability, customer service, usability)
- Clusters recurring negative complaints and surfaces actionable insights for sellers
- Presents all results through an interactive e-commerce demo website

The core challenge is **Turkish language understanding**. Turkish is an agglutinative language — words are built by stacking suffixes, and negation can be embedded inside a word or expressed implicitly through context. A keyword-based approach ("look for the word *kötü*") fails on sentences like *"Kötü diyemem, gayet iyi"* (literally "I cannot say bad, quite good" → positive). A contextual deep learning model is required.

---

2. Dataset & Data Preprocessing

2.1 Dataset

The dataset consists of 744 labeled Turkish product reviews across 8 product categories (headphones, laptop, smartwatch, robot vacuum, e-reader, game console, TV, smartphone). Reviews were manually collected and labeled as **positive** or **negative** at the sentence level.

Split	Samples
Training	595
Validation	74
Test	75
Total	**744**

To reach this size from an initial set of 40 labeled reviews, a data augmentation pipeline (`training/prepare_data.py`) was applied:

- Synonym replacement using a Turkish synonym dictionary
- Random sentence reordering within multi-sentence reviews
- Back-translation augmentation (Turkish → English → Turkish)
- Combination with a subset of the publicly available Turkish product review corpus on HuggingFace

Sentiment distribution: 52% positive, 48% negative (intentionally balanced to prevent class bias).

2.2 Preprocessing Pipeline

Raw review text goes through the following cleaning steps before being passed to the tokenizer:

```
Raw Review Text
        |
        ▼
[HTML tag removal — BeautifulSoup]
        |
        ▼
[Unicode normalization — NFKC]
   (handles ş, ç, ğ, ı, ö, ü correctly)
        |
        ▼
[Extra whitespace collapse]
        |
        ▼
[Turkish stop word filtering]
   (ve, bu, şu, gibi, için, bir, ...)
        |
        ▼
Clean Text → BERT WordPiece Tokenizer
```

Key preprocessing decisions:

- **NFKC normalization** is essential for Turkish: the character *ğ* has multiple Unicode representations and inconsistent encoding is common in user-generated content.
- **Stop word removal** reduces noise before tokenization, though BERT's subword tokenizer handles unknown tokens gracefully regardless.

• **No stemming or lemmatization** — BERT's WordPiece tokenizer handles morphological variation by splitting words into subword units, making rule-based stemming unnecessary and potentially harmful.

Implementation: `app/preprocessing/cleaner.py`

---

3. Model Architecture

3.1 Base Model: Turkish BERT

The core model is `savasy/bert-base-turkish-sentiment-cased`, a BERT model pre-trained on 200 million tokens of Turkish text and fine-tuned for binary sentiment classification.

BERT (Bidirectional Encoder Representations from Transformers) is a Transformer encoder that reads the full input sequence simultaneously in both directions, producing contextual word representations. This bidirectionality is what allows it to resolve ambiguity — the meaning of a word depends on both what comes before *and* after it.

Architecture specifications:

Component	Value
Transformer encoder layers	12
Hidden dimension	768
Attention heads per layer	12
Dimension per head	64
Feed-forward dimension	3072
Total parameters	~110 million
Vocabulary	32,000 Turkish WordPiece tokens

3.2 Forward Pass (Inference)

```
Input: "Ses kalitesi harika ama kargo çok geç geldi"
          |
          ▼
    [WordPiece Tokenizer]
    [CLS] ses kalite ##si harika ama kargo çok geç gel ##di [SEP]
          |
          ▼
    [Token Embeddings]          (32,000 × 768 lookup table)
  + [Position Embeddings]       (sequence position → 768)
  + [Segment Embeddings]        (single sentence → all zeros)
          |
          ▼
    [Transformer Block × 12]
    Each block:
      ┌─ Multi-Head Self-Attention (12 heads × 64D)
      │    "harika" attends to "ses kalitesi" → quality context
      │    "geç" attends to "kargo" → shipping context
      ├─ Add & LayerNorm
      ├─ Feed-Forward Network (768 → 3072 → 768)
      └─ Add & LayerNorm
          |
          ▼
    [CLS Token — 768D representation of entire sequence]
          |
          ▼
    [Dropout (0.1)]
          |
          ▼
    [Linear Layer: 768 → 2]   {negative, positive}
          |
          ▼
    [Softmax]
    P(negative) = 0.62,  P(positive) = 0.38
          |
          ▼
    Score = P(positive) − P(negative) = −0.24  (mildly negative)
```

3.3 Sentence-Level Aspect Analysis

Rather than scoring each review as a single unit, ReviewLens splits each review into sentences and runs BERT on each sentence independently. This enables **aspect-level separation**:

```
Review: "Ses kalitesi harika ama kargo çok geç geldi, kutu ezilmişti"
        |
        ▼ [Split on: . ! ? , ; ama fakat ancak]
        |
    Sentence 1: "Ses kalitesi harika"
    → BERT score: +0.97
    → Detected aspects: [quality]   (keyword: "kalite")
        |
    Sentence 2: "kargo çok geç geldi kutu ezilmişti"
    → BERT score: −0.61
    → Detected aspects: [shipping]  (keyword: "kargo", "kutu")
        |
        ▼
    aspect_scores:
      quality:  +0.77  (blended: 0.85 × 0.97 + 0.15 × general)
      shipping: −0.43  (blended: 0.85 × −0.61 + 0.15 × general)
      price:    −0.05  (not mentioned → general × 0.15)
      ...
```

Blending formula:

`aspect_score = w × sentence_avg + (1−w) × general_score`

where `w = min(0.85, 0.5 + n × 0.1)` and `n` = number of sentences mentioning that aspect.

This prevents a single extreme sentence from dominating the aspect score.

3.4 Fine-Tuned Model Class (for Training)

For fine-tuning, a custom PyTorch model wraps the BERT encoder with a trainable classification head:

```python
class AspectSentimentModel(nn.Module):
    def __init__(self):
        self.encoder = AutoModel.from_pretrained("savasy/bert-base-turkish-sentiment-cased")
        self.dropout = nn.Dropout(0.3)
        self.classifier = nn.Sequential(
            nn.Linear(768, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 2),    # binary: positive / negative
        )

    def forward(self, input_ids, attention_mask):
        outputs = self.encoder(input_ids=input_ids, attention_mask=attention_mask)
        cls = outputs.last_hidden_state[:, 0, :]   # [CLS] token
        return self.classifier(self.dropout(cls))
```

Implementation: `app/models/sentiment.py`

---

4. Training Process

4.1 Transfer Learning Strategy

Training a 110M-parameter Transformer from scratch requires hundreds of millions of labeled examples and weeks of GPU time. Instead, the project uses **transfer learning**:

- Start from `savasy/bert-base-turkish-sentiment-cased` — a model already trained on Turkish text that understands morphology, negation, and sentiment patterns.
- Continue training (fine-tune) on our 595 Turkish e-commerce reviews.
- The model adapts its weights toward e-commerce-specific language while retaining broad Turkish language understanding.

4.2 Loss Function

Binary Cross-Entropy loss:

```
L = −[ y · log(ŷ) + (1 − y) · log(1 − ŷ) ]
```

Where $y \in \{0, 1\}$ is the true label and $\hat{y}$ is the predicted positive probability.

The loss penalizes confident wrong predictions much more than uncertain ones, pushing the model to be calibrated.

4.3 Backpropagation

After each forward pass, gradients are computed and propagated backward through all 12 Transformer layers:

```
∇_θ L = ∂L/∂θ    (chain rule through softmax → linear → dropout → 12× attention+FFN)

θ_new = θ_old − α · m̂ / (√v̂ + ε)     [AdamW update rule]
```

AdamW optimizer maintains a running mean $\hat{m}$ and variance $\hat{v}$ of past gradients, effectively giving each weight its own adaptive learning rate. This is standard for Transformer fine-tuning because different layers require different update magnitudes — early layers (general syntax) need smaller updates than the final classification head.

4.4 Training Configuration

Hyperparameter	Value	Justification
Optimizer	AdamW	Standard for Transformer fine-tuning

Hyperparameter	Value	Justification
Learning rate	2e-5	Small enough to not destroy pre-trained weights
β□, β□	0.9, 0.999	AdamW defaults
Weight decay	0.01	L2 regularization to prevent overfitting
Epochs	3	Loss converged; more epochs caused overfitting
Batch size	8	Fits comfortably in RTX 4070 VRAM
Max sequence length	128 tokens	Covers 99% of review lengths
LR scheduler	Linear warmup + decay	Standard for BERT fine-tuning
Warmup steps	10% of total steps	Prevents instability at training start
Device	NVIDIA RTX 4070 Laptop GPU	CUDA, ~8GB VRAM

4.5 Training History

Epoch	Train Loss	Val Accuracy	Val F1	Val Precision	Val Recall
1	0.3605	94.59%	0.9459	89.74%	100.00%
2	0.1185	95.95%	0.9565	97.06%	94.29%
3	**0.0454**	**97.30%**	**0.9714**	**97.14%**	**97.14%**

Total training time: 42.3 seconds on NVIDIA RTX 4070 Laptop GPU.

The monotonically decreasing training loss confirms the model is successfully minimizing the cross-entropy objective through gradient descent. The gap between training loss (0.045) and validation accuracy (97.3%) is small, indicating no significant overfitting.

---

5. Evaluation Metrics

5.1 Classification Metrics (Fine-Tuning)

The model was evaluated on a held-out test set of 75 samples (never seen during training or validation):

Metric	Score
Accuracy	**94.67%**

Metric	Score
F1 Score	**95.00%**
Precision	95.00%
Recall	95.00%

Metric definitions:

- **Accuracy** = (TP + TN) / Total — fraction of correct predictions
- **Precision** = TP / (TP + FP) — of predicted positives, how many were actually positive
- **Recall** = TP / (TP + FN) — of actual positives, how many were correctly identified
- **F1** = 2 × (Precision × Recall) / (Precision + Recall) — harmonic mean, robust to class imbalance

The balanced F1 and Recall (both 95%) confirm the model performs equally well on both positive and negative reviews without systematic bias toward either class.

5.2 Sentiment Score Range

Each review receives a continuous sentiment score in **[−1.0, +1.0]**:

```
score = P(positive) - P(negative)

+1.0  →  Strongly positive  ("Mükemmel ürün, çok memnun kaldım")
 0.0  →  Neutral             (balanced or ambiguous)
-1.0  →  Strongly negative  ("Berbat ürün, hiç tavsiye etmem")
```

5.3 System-Level Results (8 Demo Products)

The full pipeline was tested against all 8 products in the demo dataset:

Product	Reviews	Positive	Negative	Overall Score	Verdict
SoundMax Pro X Kulaklık	10	9	1	**+0.79**	Çok Olumlu
ProBook Air 14 Laptop	10	1	9	**−0.80**	Çok Olumsuz
FitMax 7 Pro Akıllı Saat	10	5	5	**−0.14**	Karışık
CleanBot V8 Robot Süpürge	9	8	1	**+0.85**	Çok Olumlu
InkPaper E-Kitap Okuyucu	9	0	9	**−0.89**	Kritik Olumsuz

Product	Reviews	Positive	Negative	Overall Score	Verdict
PoyStation 5 Oyun Konsolu	8	3	5	**−0.21**	Karışık
VisionX 4K Ultra TV	8	0	8	**−0.87**	Kritik Olumsuz
NovaPhone 14 Pro	16	5	11	**−0.51**	Olumsuz

All scores match the expected sentiment given the review content (manually verified). The model correctly handles:

- **Negation:** *"Kötü diyemem, gayet iyi"* → +0.97 □
- **Mixed reviews:** *"Ses kalitesi harika ama kargo çok geç geldi"* → quality +0.97, shipping −0.61 □
- **Sarcasm / strong negative:** *"Harika kargo, kutuyu paramparça etmişler"* → −0.88 □

5.4 Aspect Score Example

SoundMax Pro X (10 reviews, 9 positive / 1 negative):

Aspect	Score	Interpretation
Quality	+0.97	Customers love the sound quality
Price	+0.92	Considered good value
Durability	+0.89	Battery and build quality praised
Usability	+0.89	App and controls intuitive
Customer Service	+0.79	No specific complaints
Shipping	**−0.06**	**One complaint: late delivery, damaged box**

The system correctly isolated the single negative review about shipping and separated it from the otherwise positive quality scores.

---

6. System Architecture

6.1 Technology Stack

Layer	Technology	Purpose
NLP Model	HuggingFace Transformers + PyTorch	BERT inference and fine-tuning

Layer	Technology	Purpose
Clustering	sentence-transformers + HDBSCAN	Negative review topic extraction
API	FastAPI + Uvicorn	Async REST API server
Schema	Pydantic v2	Request/response validation
Frontend	HTML5 + CSS3 + Vanilla JS	Interactive demo site
Hardware	NVIDIA RTX 4070 Laptop GPU	CUDA-accelerated inference

6.2 Pipeline Flow

```
User submits reviews via browser
            |
            ▼
    POST /api/v1/analyze
            |
            ▼
    [1. Text Cleaning]
    Unicode normalization, HTML removal, whitespace
            |
            ▼
    [2. BERT Inference — GPU]
    Per-sentence tokenization → forward pass → score □ [-1, 1]
    12 Transformer layers × each sentence
            |
            ▼
    [3. Aspect Aggregation]
    Keyword detection → weighted blending → 6 aspect scores
            |
            ▼
    [4. Negative Review Clustering — HDBSCAN]
    sentence-transformers embeddings → density-based clustering
    → Top recurring issues + outlier detection
            |
            ▼
    [5. Report Generation]
    Pros/Cons, buyer summary, seller recommendations, red flags
            |
            ▼
    JSON Response → Frontend renders visualizations
```

6.3 API Endpoints

```
POST /api/v1/analyze
  Input:  { product_name, reviews: [str] }
  Output: { overall_sentiment, aspect_scores, sentiment_breakdown,
            top_issues, pros, cons, red_flags, buyer_summary,
            seller_report, confidence, review_count }

POST /api/v1/analyze/batch
  Input:  { products: [{ product_name, reviews }] }
  Output: { results: [...], total_products, total_reviews }

GET  /api/v1/health
  Output: { status, version, models_loaded }
```

6.4 Project Structure

```
reviewlens/
├── app/
│   ├── main.py                 # FastAPI application entry point
│   ├── api/routes.py           # API endpoint definitions + auth
│   ├── services/analyzer.py    # Analysis pipeline orchestrator
│   ├── models/
│   │   ├── sentiment.py        # Turkish BERT analyzer + AspectSentimentModel
│   │   ├── topic_extractor.py  # HDBSCAN negative review clustering
│   │   └── summarizer.py       # Pros/cons, buyer summary, seller report
│   ├── preprocessing/
│   │   └── cleaner.py          # Text cleaning pipeline
│   └── schemas/models.py       # Pydantic request/response schemas
├── demo_site/
│   ├── index.html              # TechMart frontend (9 products + sandbox)
│   └── images/                 # Product images
├── models/
│   └── finetuned_bert/         # Fine-tuned model weights (safetensors format)
├── training/
│   ├── prepare_data.py         # Data augmentation → 744 samples
│   ├── finetune.py             # BERT fine-tuning script (GPU)
│   ├── use_finetuned.py        # Model integration helper
│   └── results.json            # Training metrics log
├── data/
│   └── labeled_reviews.json    # Initial labeled dataset
├── requirements.txt
├── .env                        # Environment config (model path, API keys)
└── README.md
```

---

7. Course Concepts Integration

Transformer Architecture □

BERT's 12-layer encoder with multi-head self-attention is the backbone of the system. Each attention head learns to focus on different linguistic relationships — negation, aspect-word associations, syntactic dependencies.

Attention Mechanisms

Self-attention allows each token to attend to all other tokens simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d\_k}) \cdot V$$

This is what enables *"Kötü diyemem"* to be understood as positive — the token *kötü* (bad) attends to *diyemem* (I cannot say) and the combined representation reflects negation.

Embeddings

Three types of embeddings are used:

- **Token embeddings:** 768D lookup for each WordPiece token (learned during pre-training)
- **Positional embeddings:** Encode word order in the sequence
- **Contextual embeddings:** BERT's hidden states (each token's 768D representation changes based on context)
- **Sentence embeddings:** `all-MiniLM-L6-v2` produces fixed 384D sentence vectors for HDBSCAN clustering

Gradient Descent & Backpropagation

Fine-tuning explicitly performed gradient descent over 3 epochs. Training loss decreased from 0.3605 → 0.0454, directly demonstrating convergence through backpropagation across 12 Transformer layers.

Modern Frameworks

- **PyTorch:** Tensor computation, autograd, model definition
- **HuggingFace Transformers:** Model loading, tokenization, Trainer API
- **FastAPI:** Production-grade async API server

---

8. How to Run

```
# Prerequisites: Python 3.11, NVIDIA GPU (optional but recommended)

# 1. Clone the repository
git clone https://github.com/KeremOzcn/reviewlens.git
cd reviewlens

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate        # Windows
# source venv/bin/activate   # Linux/Mac

# 3. Install dependencies
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt

# 4. Start the API server (Terminal 1)
uvicorn app.main:app --host 127.0.0.1 --port 8001

# 5. Start the frontend server (Terminal 2)
cd demo_site
python -m http.server 8080

# 6. Open browser → http://localhost:8080
```

Note: The fine-tuned model weights (`models/finetuned_bert/model.safetensors`, 440MB) are not included in the repository due to GitHub file size limits. To reproduce fine-tuning:

```
python training/prepare_data.py   # generate training data
python training/finetune.py       # fine-tune (~42s on RTX 4070)
python training/use_finetuned.py  # integrate into system
```

Alternatively, the base model `savasy/bert-base-turkish-sentiment-cased` will be downloaded automatically from HuggingFace Hub on first run.

---

9. Conclusion

ReviewLens demonstrates a complete NLP application pipeline — from raw Turkish text to structured product intelligence. The key contributions are:

- **Fine-tuned Turkish BERT** achieving 94.67% test accuracy on e-commerce sentiment classification
- **Aspect-level sentence analysis** that separates shipping complaints from quality praise within the same review
- **Integrated clustering pipeline** (HDBSCAN + sentence-transformers) that groups recurring negative complaints without requiring predefined categories
- **End-to-end production system** with a FastAPI backend and interactive frontend

The system correctly handles the core challenges of Turkish NLP: morphological variation, negation, and mixed-sentiment reviews — all through the contextual understanding provided by the Transformer architecture.

References

• Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805*

• Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS 2017)*

• Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*

• Campello, R.J.G.B., Moulavi, D., & Sander, J. (2013). Density-Based Clustering Based on Hierarchical Density Estimates. *PAKDD 2013*

• Savasy Turkish BERT Model: https://huggingface.co/savasy/bert-base-turkish-sentiment-cased

• HuggingFace Transformers Documentation: https://huggingface.co/docs/transformers/